Q3.

CLAIM: RB-ENUMERATE$(u, low, high)$ correctly outputs all keys k such that $low \le k \le high$ in strictly increasing order

PROOF:

The algo relies on a modified in-order traversal of RB-Tree. Standard inorder traversal visits the node in monotonically increasing order (∵ RB trees maintain BST property).

∴ the algo prunes subtrees based on:

Case 1: $u.key \le low$ – algo restricts traversal to left subtree in left subtree

∵ All descendants $< u.key$ ⇒ no valid keys are missed. ~~∴ algo restricts traversal to left subtree~~

Case 2: $u.key \ge high$ – algo restricts traversal to right subtree

∵ All descendants in right subtree $> u.key$ ⇒ no valid keys are missed ∴ ~~algo~~

Case 3: $low \le u.key \le high$ ⇒ node satisfies condition & is printed

Algo processes left subtree, prints current node, then process right subtree maintaining the exact in-order sequence. //.

TIME COMPLEXITY ANALYSIS

$2^h = n+1$
$\Rightarrow h = O(\log(n+1))$

Height of RB tree : $O(\log(n+1))$

⇒ For each lower bound & high bound, max depth of path reqd = ~~$O(\log n)$~~ $O(\log(n+1))$

For any node outside the range

(I) condition fails & returns in $O(1)$

(II) lies on the boundary path ⇒ $O(\log n)$

For each of the r nodes, within the valid range, the algo performs $O(1)$ work to evaluate & print the node

∴ Total time reqd. = $O(r + \log(n+1))$ //.